trie has more nodes, the more this overhead is, the larger the delta will be.

Obviously, however, the shape of the trees and the actual number of nodes also play a big role: in the example in figure 6.3 only a short prefix is shared among the keys. Turns out, tries are more efficient when holding keys with a large shared prefix: the example in figure 6.4 shows how the balance can be in favor of tries in these cases; as you can see, in the example of figure 6.4 most trie nodes are black (key nodes) and when the ratio of key nodes versus intermediate nodes is higher, intuitively it means that the efficiency of the trie is also higher, because most nodes contribute to storing words that, in a BST, would require a full string (and BST node) to be stored.

In the end, in this second example, with just nine words stored, using the same assumptions as above, the difference becomes 220 bytes versus 134 bytes, with tries allowing to save almost 40% of the space; if we consider an 8-byte overhead for nodes, the difference would be 256 versus 178 bytes, and the saving would be around 30% - still impressive, for large trees, containing dictionaries or indexing large texts, we would talk about even hundreds of megabytes.